

**BEATRIZ DOS SANTOS
BRENDA RIBEIRO ALVES
MARIA EDUARDA BEZERRA**

PROJETO INTEGRADOR - 3º TERMO

Manutenção Industrial 4.0

LIMEIRA – SP

2026

Sumário

1. LEVANTAMENTO DE REQUISITOS.....	1
1.1. REQUISITOS FUNCIONAIS.....	1
1.2. REQUISITOS NÃO FUNCIONAIS	1
2. PRIORIZAÇÃO.....	2
3. FIGMA.....	4
4. METODOLOGIAS ÁGEIS.....	5
4.1. METODOLOGIA KANBAN	5
5. README E VERSIONAMENTO (GIT)	6
6. DIAGRAMA.....	7
6.1. DIAGRAMA DE CLASSE	7
6.2. DIAGRAMA DE SEQUÊNCIA.....	8
6.3. DIAGRAMA BANCO DE DADOS.....	9
7. SOBRE O SISTEMA	10
7.1. VISÃO GERAL.....	10
7.2. TECNOLOGIAS E INFRAESTRUTURA.....	10
7.3. ENTIDADES PRINCIPAIS (MODELAGEM ORM).....	11
7.4. FLUXO PRINCIPAL (WORKFLOW).....	11
8. ENGENHARIA DE COMPONENTES-CHAVE.....	12
8.1. CRIAÇÃO, EXECUÇÃO E PERSISTÊNCIA DE CHECKLISTS	12
8.2. MECANISMO DE FALLBACK DE TEMPLATES	12
8.3. STATE MACHINE PARA OCORRÊNCIAS	13
8.4. AUTOMAÇÕES DE BACKGROUND	13
8.5. MAPA INTERATIVO DE MÁQUINAS	13
9. RELATÓRIOS E INDICADORES	14
9.1. INDICADORES CRÍTICOS DE MANUTENÇÃO	14
9.2. MECANISMO DE RELATÓRIOS ESTRUTURADOS	14
9.3. CONTROLE DE ACESSO E PAINÉIS CUSTOMIZADOS (RBAC).....	14
10. CONCLUSÃO	15

MANUTENÇÃO INDUSTRIAL

Situação Problema: "Na era da Indústria 4.0, a disponibilidade das máquinas é crucial para a produtividade. Paradas não planejadas custam milhares de reais e atrasam toda a cadeia produtiva. Fábricas modernas buscam migrar de anotações em papel para sistemas digitais que prevejam e registrem manutenções de forma eficiente." "O Senai contratou sua consultoria para acabar com as 'fichas de papel' penduradas nas máquinas. Eles precisam de um sistema centralizado onde a equipe de manutenção possa registrar intervenções e os gerentes possam acompanhar a saúde dos equipamentos em tempo real."

Desafio: "Sua equipe deve implementar o Back-End do 'MaintSys', uma API para controle de manutenção de maquinário industrial. O sistema deve garantir a integridade dos dados e permitir que técnicos registrem atividades através de futuros tablets ou terminais industriais."

Funcionalidades Essenciais:

1. Gestão de Técnicos e Acesso: Cadastro de técnicos especializados com autenticação segura.
2. Inventário de Máquinas: Cadastro de equipamentos (Número de Série, Modelo, Localização no Galpão, Data de Instalação).
3. Ordens de Serviço (O.S.): Criação de ordens de manutenção (Preventiva ou Corretiva), vinculando um técnico a uma máquina específica.
4. Histórico de Manutenções: Log completo de todas as intervenções realizadas em uma máquina, permitindo análise de reincidência de defeitos.
5. Alertas de Status: Sistema para notificar quando uma máquina muda de status (ex: "Máquina 04 em Parada Crítica" ou "Manutenção Concluída").

Requisitos e Restrições:

- Back-End desenvolvido em Laravel (PHP).
- Banco de dados relacional MySQL.
- Utilizar o Eloquent ORM para modelar as tabelas e relações (ex: Uma Máquina tem muitas Ordens de Serviço).
- API RESTful retornando JSON.
- Código seguindo padrões PSR e Clean Code.
- Publicação no GitHub e testes via Insomnia/Postman.

1. LEVANTAMENTO DE REQUISITOS

O levantamento de requisitos é o processo de identificar, analisar e documentar as necessidades reais dos clientes e usuários.

1.1. REQUISITOS FUNCIONAIS

Especificam as interações e os serviços que o sistema deve oferecer para atender às necessidades do usuário.

RF1 O sistema deve coletar dados de sensores (vibração, temperatura e pressão) em tempo real.
RF2 O sistema deve armazenar os dados coletados em banco de dados histórico.
RF3 O sistema deve analisar automaticamente os dados utilizando algoritmos preditivos.
RF4 O sistema deve gerar alertas automáticos quando detectar anomalias.
RF5 O sistema deve permitir o cadastro de máquinas e equipamentos.
RF6 O sistema deve permitir o cadastro de usuários com diferentes níveis de acesso.
RF7 O sistema deve gerar relatórios de desempenho e histórico de falhas.
RF8 O sistema deve enviar notificações por e-mail ou aplicativo móvel. (Versão - Mobile)
RF9 O sistema deve permitir agendamento de manutenções preventivas.
RF10 O sistema deve exibir dashboard com indicadores (MTBF, MTTR, disponibilidade). (Versão - Mobile)

1.2. REQUISITOS NÃO FUNCIONAIS

Estabelecem os critérios que restringem ou qualificam a operação do sistema, garantindo sua viabilidade e experiência.

RNF1 Desempenho: O sistema deve processar e exibir dados em até 3 segundos após a coleta.
RNF2 Disponibilidade: O sistema deve estar disponível 99% do tempo.
RNF3 Segurança: O sistema deve possuir autenticação por login e senha criptografada.
RNF4 Escalabilidade: O sistema deve suportar no mínimo 500 equipamentos cadastrados.
RNF5 Usabilidade: A interface deve ser intuitiva e permitir treinamento em no máximo 4 horas.
RNF6 Confiabilidade: Os dados devem possuir backup automático diário.
RNF7 Compatibilidade: O sistema deve funcionar em navegadores atualizados (Chrome, Edge e Firefox).
RNF8 Manutenibilidade: O sistema deve permitir atualização de software sem interromper o funcionamento por mais de 10 minutos.

2. PRIORIZAÇÃO

O método MoSCoW é uma técnica de priorização que organiza requisitos em quatro níveis. Ele serve para gerenciar o foco e alinhar expectativas com os stakeholders, evitando o desperdício de recursos enquanto o projeto ainda não está pronto.

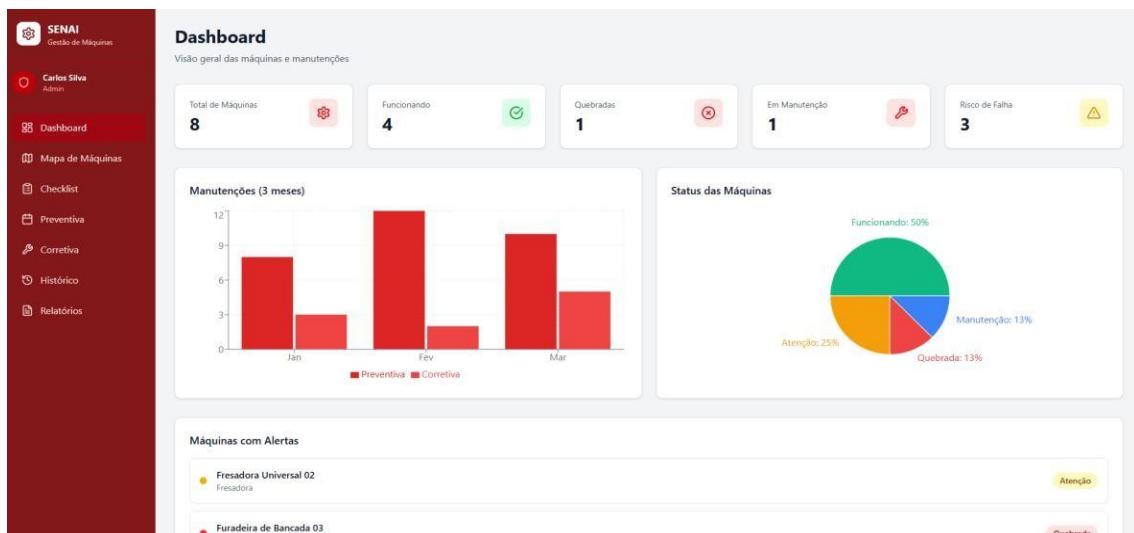
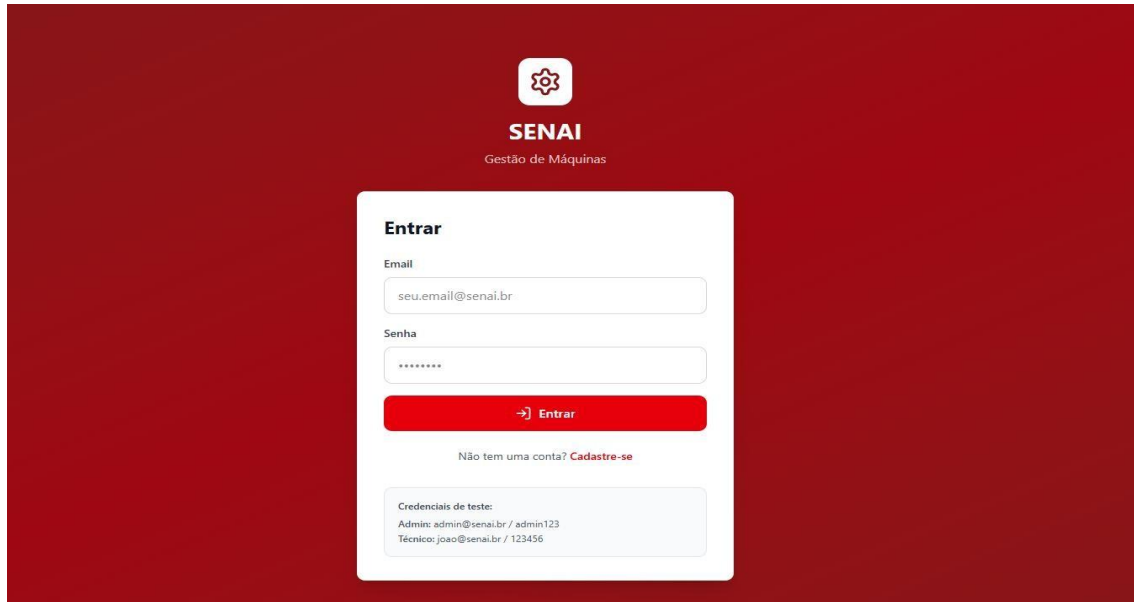
- **MUST HAVE (Deve ter):** Crítico. Sem isso, o projeto não é entregue ou não funciona. Essencial.
- **SHOULD HAVE (Deveria ter):** Importante, mas não vital. Pode ser entregue logo após o essencial. Alta Prioridade.
- **COULD HAVE (Poderia ter):** Desejável. É um diferencial que só será feito se sobrar tempo/recurso. Opcional.
- **WON'T HAVE (Não terá):** Fora de escopo para este momento. Ajuda a focar no que importa agora. Futuro.

Requisito	Priorização	Explicação
RF1 – Coletar dados de sensores em tempo real	M. H	Essencial para o funcionamento do sistema de monitoramento preditivo.
RF2 – Armazenar dados em banco histórico	M. H	Necessário para análises, rastreabilidade e geração de relatórios.
RF3 – Análise automática com algoritmos preditivos	M. H	Principal diferencial do sistema de manutenção preditiva.
RF4 – Gerar alertas automáticos de anomalias	M. H	Fundamental para evitar falhas críticas e reduzir tempo de parada.
RF5 – Cadastro de máquinas e equipamentos	M. H	Base estrutural para organização e monitoramento dos ativos.
RF6 – Cadastro de usuários com níveis de acesso	M. H	Essencial para segurança e controle de permissões.
RF7 – Gerar relatórios de desempenho e falhas	S. H	Importante para análise estratégica e tomada de decisão.
RF8 – Enviar notificações por e-mail/app	S. H	Agrega agilidade na comunicação de eventos críticos.
RF9 – Agendamento de manutenções preventivas	S. H	Complementa a estratégia de manutenção preditiva.
RF10 – Dashboard com indicadores (MTBF, MTTR, disponibilidade)	S. H	Importante para visualização gerencial e acompanhamento de desempenho.

RNF1 – Processamento em até 3 segundos	M. H	Requisito crítico de desempenho para garantir resposta em tempo real.
RNF2 – Disponibilidade de 99%	M. H	Garante confiabilidade mínima para operação industrial.
RNF3 – Autenticação com senha criptografada	M. H	Requisito essencial de segurança da informação.
RNF4 – Suportar 500 equipamentos	S. H	Importante para escalabilidade inicial do sistema.
RNF5 – Interface intuitiva (treinamento até 4h)	S. H	Melhora adoção e reduz custos de capacitação.
RNF6 – Backup automático diário	M. H	Essencial para garantir integridade e recuperação de dados.
RNF7 – Compatível com Chrome, Edge e Firefox	M. H	Importante para acessibilidade, mas não crítico na fase inicial.
RNF8 – Atualização com interrupção máxima de 10 min	C. H	Desejável para continuidade operacional, mas pode ser evoluído posteriormente.

3. FIGMA

Foi a peça-chave para validar a experiência do usuário (UX) e a interface (UI) antes do código. O protótipo de alta fidelidade no Figma garantiu o alinhamento visual com a marca SENAI, eliminou erros de usabilidade e acelerou o desenvolvimento ao servir como guia exato para a implementação técnica.



Você pode visualizar o layout completo através deste link: [FIGMA](#)

4. METODOLOGIAS ÁGEIS

São formas de gerenciar projetos focadas em entregas rápidas, adaptabilidade a mudanças e trabalho em equipe constante.

4.1. METODOLOGIA KANBAN

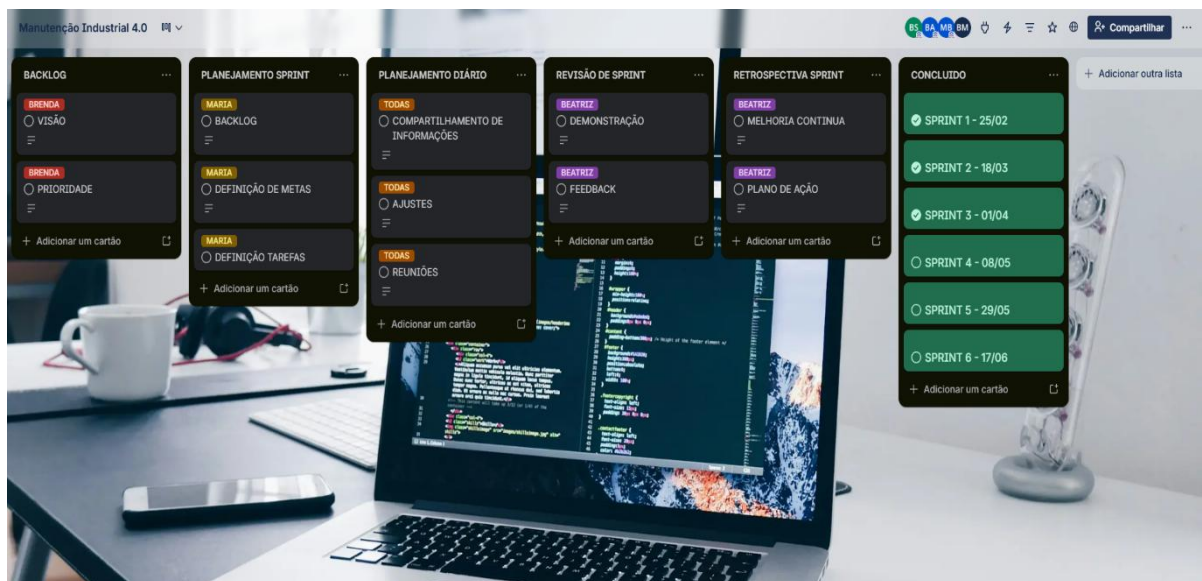
O Kanban é um método de gestão visual focado na eficiência do fluxo de trabalho.

Utilizando cartões e colunas (como "A Fazer", "Em Execução" e "Feito"), ele permite que a equipe visualize o status de cada tarefa em tempo real.

O grande diferencial é a aplicação de limites de WIP (Work In Progress), que impedem a sobrecarga da equipe e garantem que o foco esteja na conclusão, não apenas no início de novas demandas.

Vantagens: Alta flexibilidade para mudanças de prioridade e transparência total do processo, facilitando a identificação imediata de gargalos.

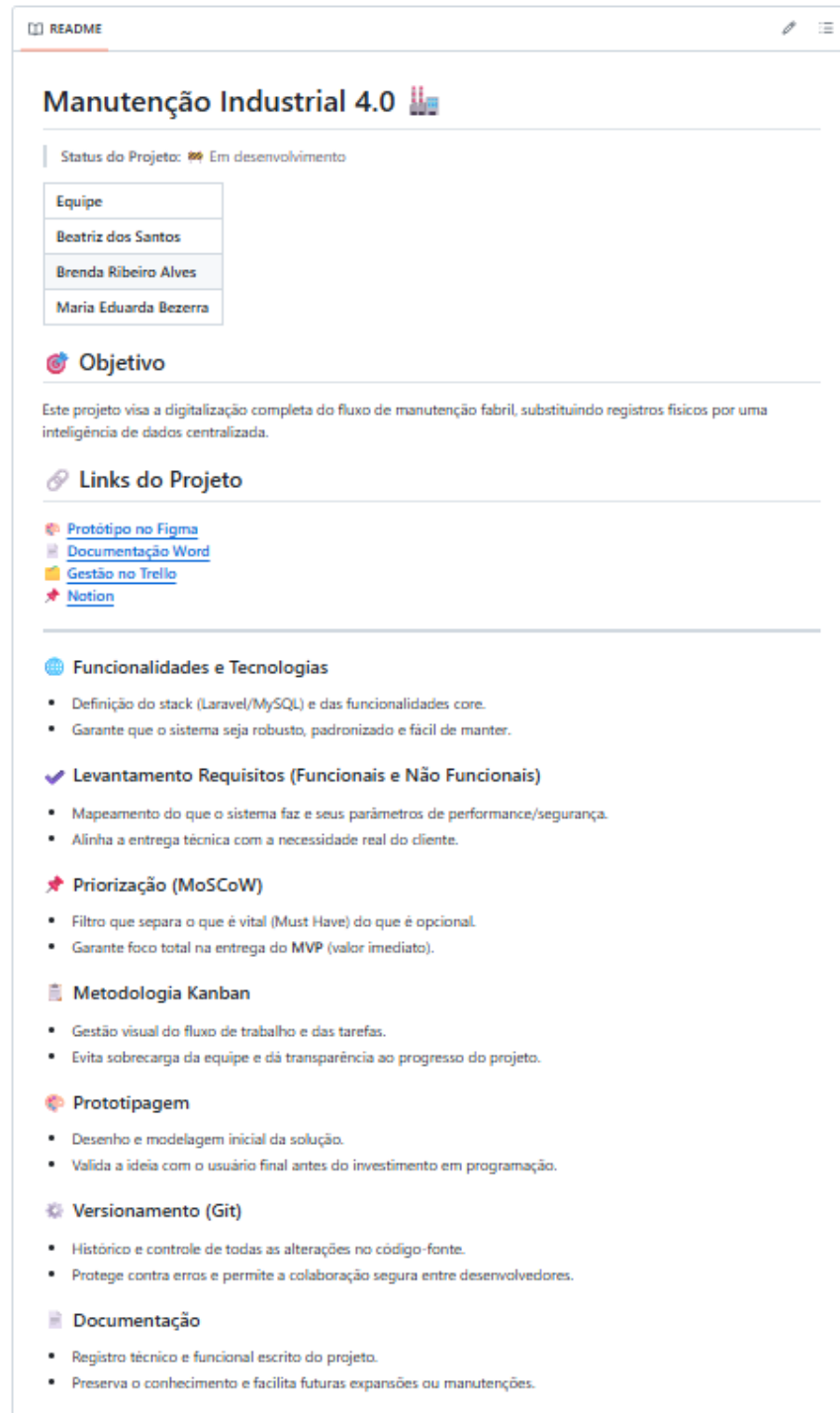
Desvantagens: A ausência de prazos rígidos (sprints) pode dificultar a estimativa de grandes projetos a longo prazo. Além disso, exige alta disciplina da equipe para que o quadro reflita a realidade.



Clique [Aqui](#) para abrir o documento original

5. README E VERSIONAMENTO (GIT)

No desenvolvimento do sistema para o Senai, a transição do físico para o digital exige clareza e segurança. Para garantir que o projeto seja sustentável e profissional, utilizamos duas ferramentas fundamentais: o arquivo README e o Versionamento (Git).

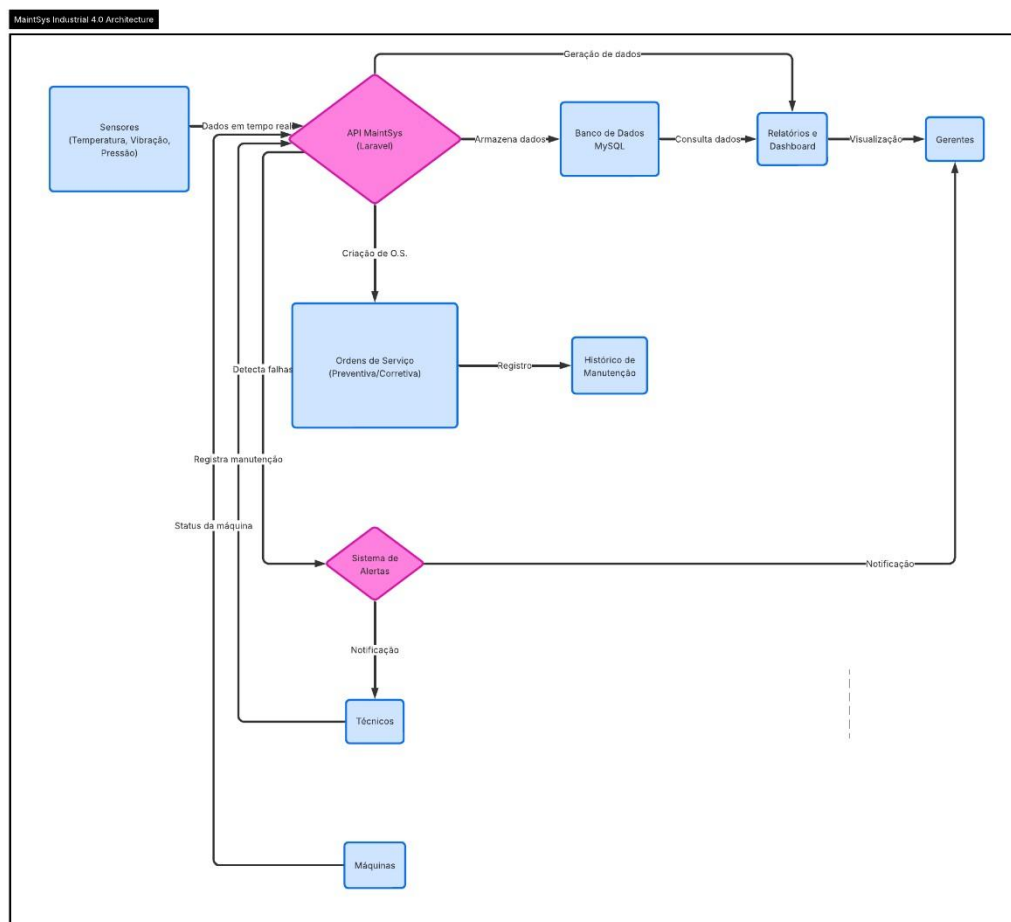


6. DIAGRAMA

Um diagrama é uma representação gráfica e esquemática de informações. Ele utiliza simbologia padronizada para descrever estruturas, fluxos de trabalho ou interações entre componentes de um sistema.

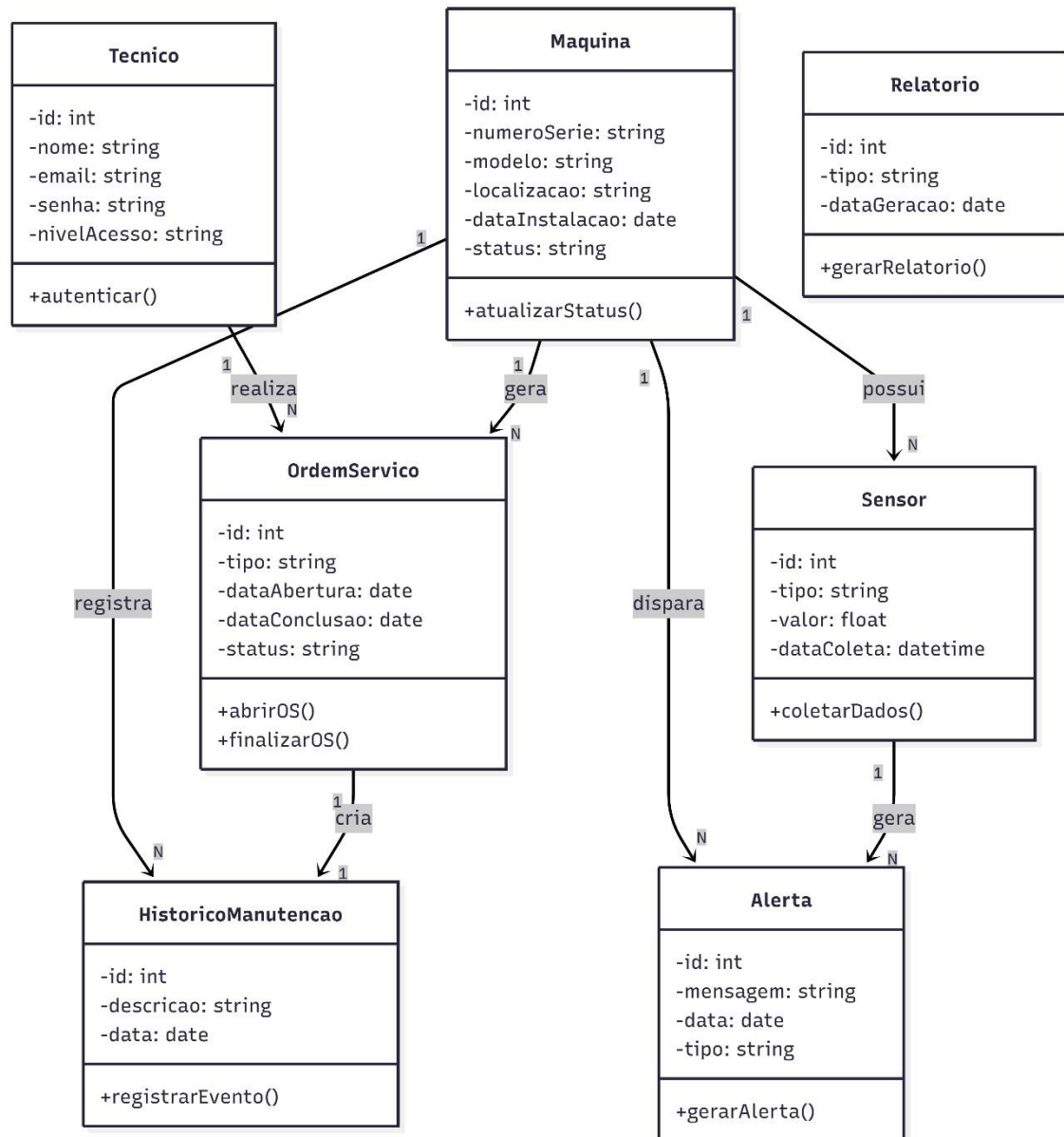
6.1. DIAGRAMA DE CLASSE

Este diagrama representa o ecossistema da solução, ilustrando o fluxo de dados desde a captura física nos sensores até a tomada de decisão gerencial.



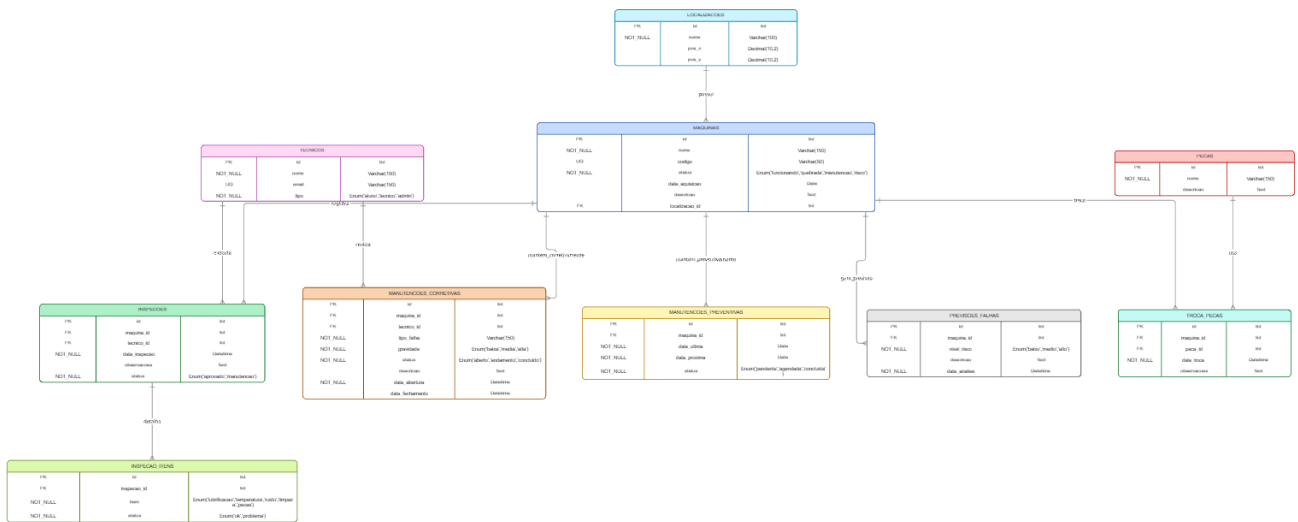
6.2. DIAGRAMA DE SEQUÊNCIA

O diagrama detalha a estrutura interna do sistema, definindo as propriedades e responsabilidades de cada objeto dentro da aplicação Laravel.



6.3. DIAGRAMA BANCO DE DADOS

Serve para visualizar, planejar e documentar a estrutura de dados de um sistema, mapeando tabelas, colunas e seus relacionamentos.



Para conferir o detalhamento desse diagrama, acesse o link: [MIRO](#)

7. SOBRE O SISTEMA

O MaintSys é uma aplicação web de alta performance desenvolvida com Laravel 12 + Filament 3 (utilizando componentes nativos do Livewire) para o gerenciamento e controle completo de manutenção industrial.

O sistema atua como o núcleo operacional e estratégico, centralizando o cadastro de ativos, a automação de fluxos de trabalho e o fornecimento de inteligência de negócios através de indicadores e relatórios estruturados.

7.1. VISÃO GERAL

O objetivo principal do MaintSys é acompanhar e auditar todo o ciclo de vida da manutenção industrial. O sistema modela e controla nativamente:

- **Gestão de Ativos:** Cadastro e monitoramento de Máquinas (Máquina) vinculadas a um Setor (Setor), com tipificação (TipoMaquina), status operacional (StatusMaquina), dados de localização geográfica/espacial dentro da planta e parâmetros de manutenção preventiva.
- **Manutenção por Eventos:** Fluxo reativo e proativo baseado em Ocorrências (Ocorrencia), Inspeções (Inspecao), Corretivas (Corretiva) e preventivas (Preventiva).
- **Suporte Logístico:** Estrutura para Solicitação de Peças (SolicitacaoPeca) e Orçamentos (Orcamento) para gerenciar o tempo de espera e compras de insumos.

7.2. TECNOLOGIAS E INFRAESTRUTURA

A arquitetura de tecnologia foi selecionada para garantir robustez, auditabilidade e escalabilidade:

- **Backend:** Laravel 12 (PHP 8.x+)
- **Painel Administrativo e Interface:** Filament 3 & Livewire (SPA-like UX)
- **Controle de Acesso (RBAC):** Spatie Laravel Permission
- **Banco de Dados:** MySQL (Relacional, gerenciado integralmente via *Migrations* do ecossistema Laravel)
- **Armazenamento de Arquivos:** Upload de evidências multimídia diretamente para buckets Amazon S3 (ou Supabase Storage compatível com S3), utilizando abstração nativa `Storage::disk('s3')`.

7.3. ENTIDADES PRINCIPAIS (MODELAGEM ORM)

- Setor: Áreas físicas e organizacionais da empresa (ex: Usinagem, CNC, Solda).
- Máquina: Equipamentos industriais contendo dados técnicos, coordenadas de posicionamento (`posicao_x`, `posicao_y`) e campos de controle de preventiva (`ultima_preventiva`, `periodicidade_preventiva`).
- Ocorrência: Entidade centralizadora que encapsula e rastreia o estado de uma falha ou chamado ao longo do fluxo.
- Inspeção: Diagnóstico técnico inicial atrelado a uma ocorrência que determina as ações subsequentes com base em checklists.
- Corretiva: Registro de manutenção executada para sanar uma falha ativa, computando custos de reparo e tempo de indisponibilidade.
- Preventiva: Intervenção planejada baseada em periodicidade cronológica.
- Checklist (Modelo, Pergunta e Resposta): Estrutura flexível e polimórfica que permite aplicar listas de verificação em Inspeções, Corretivas ou Preventivas.
- Histórico: Registro global de auditoria automatizado utilizando relacionamento polimórfico (`morphMany`), mapeando eventos e ações cruciais de usuários no sistema.

7.4. FLUXO PRINCIPAL (WORKFLOW)

1. Cadastro de setores e parametrização de tipos de máquinas.
2. Cadastro de máquinas com vinculação a templates de checklists obrigatórios.
3. Abertura de Ocorrências devido a falhas reportadas na planta.
4. Execução e triagem através de uma Inspeção Técnica baseada em perguntas.
5. Direcionamento automático (State Machine) para o fluxo de corretiva ou Aguardando Peças/Orçamento.
6. Execução das ações de reparo, inserção de custos/tempo e fechamento do ciclo.
7. Alimentação automática do histórico de auditoria e cálculo de indicadores (MTBF/MTTR).

8. ENGENHARIA DE COMPONENTES-CHAVE

8.1. CRIAÇÃO, EXECUÇÃO E PERSISTÊNCIA DE CHECKLISTS

Gerenciado pela camada de serviço ChecklistService, o ciclo possui regras rígidas de validação:

- Geração Automatizada: No cadastro ou edição de uma Máquina (via *Filament Resource*), existem marcadores para determinar quais checklists devem ser gerados (*Inspeção, Corretiva, Preventiva*). Regra Crítica: Se os checklists de *Corretiva* ou *Preventiva* forem selecionados, o sistema força a obrigatoriedade da inclusão do checklist de *Inspeção* como base. O gatilho dispara o método ChecklistService::criarChecklistsSelecionados().
- Execução Dinâmica: As respostas do operador são padronizadas em um *Enum* (Conforme, Não Conforme, Não Aplicável).
- Validação Restrita de Não Conformidades: Caso o operador selecione a opção Não Conforme, o sistema abre obrigatoriamente uma janela modal exigindo:
 1. Descrição detalhada do problema.
 2. Evidência fotográfica (upload direto para o S3). O método ChecklistService::salvarRespostas() valida que 100% das perguntas foram respondidas. Caso haja uma resposta Não Conforme desprovida de foto ou descrição, uma ValidationException é lançada, impedindo a gravação do registro (executado via updateOrCreate por checklist_pergunta_id).

8.2. MECANISMO DE FALLBACK DE TEMPLATES

Para mitigar falhas operacionais na ausência de parametrizações específicas, os métodos templateParaMaquina() e modeloParaMaquina() operam sob uma lógica de cascata:

1. Busca um checklist modelo específico atribuído ao ID do tipo da máquina (tipo_maquina_id).
2. Caso não localizado, o sistema recorre a um template genérico (onde tipo_maquina_id é nulo).
3. Se nenhum padrão for encontrado, o sistema instancia preventivamente um checklist "vazio" (sem perguntas pré-definidas), permitindo a continuidade do fluxo de manutenção sem travar o software.

8.3. STATE MACHINE PARA OCORRÊNCIAS

Gerenciado de forma isolada através do `OcorrenciaStateMachineService`, o status de uma ocorrência não é alterado diretamente de forma arbitrária pela interface.

Ao finalizar o processo no método `finalizarAposInspecao()`, o serviço executa:

- Validação de integridade para confirmar se a inspeção pertence àquela ocorrência e se está devidamente finalizada.
- Análise da condicional `inspecao->necessita_peca`. Se houver necessidade de insumos, o status migra para `AguardandoOrcamento`; caso contrário, o fluxo avança para corretiva. As transições seguem regras centralizadas em `OcorrenciaService::transicionarStatus()`.

8.4. AUTOMAÇÕES DE BACKGROUND

O sistema conta com comandos de console agendados para a manutenção preventiva e segurança de dados:

- `GerarPreventivasFuturas`: Calcula automaticamente a próxima data de preventiva com base na data de conclusão da última manutenção e na periodicidade cadastrada.
- `MarcarPreventivasAtrasadas`: Varre o banco de dados e altera o status das manutenções programadas que ultrapassaram a janela de atendimento.
- `BloquearUsuariosExpirados`: Rotina de segurança para expiração e bloqueio de credenciais de acesso temporárias ou inativas.

8.5. MAPA INTERATIVO DE MÁQUINAS

A página customizada `Filament\Pages\MapaMaquinas` interage diretamente com o `MapaMaquinasService` para entregar uma visão geoespacial em tempo real da fábrica:

- Filtra e renderiza dinamicamente as máquinas de acordo com o setor selecionado (`setorSelecionado`).
- Provê um resumo visual dos status operacionais da planta.
- Arrastar e Soltar (Drag and Drop): Quando o administrador reposiciona um ícone de máquina na interface gráfica do mapa, o frontend emite um evento máquina-movida, acionando o método backend `atualizarPosicao($maquinaId, $x, $y)`, persistindo as novas coordenadas em campos do tipo *float* no banco de dados e recarregando a listagem em tempo real.

9. RELATÓRIOS E INDICADORES

9.1. INDICADORES CRÍTICOS DE MANUTENÇÃO

Calculados dinamicamente via `IndicadorService`:

- Disponibilidade: Percentual de tempo em que o ativo esteve apto a produzir, derivado das métricas de MTBF e MTTR.
- MTBF (Mean Time Between Failures): Tempo médio decorrido entre falhas mecânicas/operacionais.
- MTTR (Mean Time To Repair): Tempo médio necessário para a equipe técnica executar o reparo e solucionar a ocorrência.

9.2. MECANISMO DE RELATÓRIOS ESTRUTURADOS

A página `Filament\Pages\RelatorioManutencao` consolida dados operacionais e financeiros através de filtros avançados por período (`dataInicio` e `dataFim`), setor e máquina.

O motor de busca coleta e agrega dados cruzados de:

- Ocorrência carregando relacionamentos complexos (máquina.setor, supervisor/professor, técnico, inspeção).
- Corretiva contabilizando tempo de reparo (`tempo_reparo`) e custos.
- Preventiva mapeando custos e execuções agendadas.

O sistema gera sumários contendo a contagem volumétrica de chamados e o Custo Total da Operação (calculado através do somatório de `custoTotal` das corretivas com o custo das preventivas).

9.3. CONTROLE DE ACESSO E PAINÉIS CUSTOMIZADOS (RBAC)

A interface do `Filament\Pages\Dashboard` adapta seus componentes (widgets, cards informativos e gráficos) baseando-se no perfil ativo do usuário (`Auth::user()`) persistido via sessões do `Spatie Laravel Permission`:

- Admin: Acesso irrestrito. Exibe painéis macro de custos de manutenção, gráficos consolidados de MTBF/MTTR e alertas críticos do sistema.
- Supervisor (ou professor/instrutor): Visão tática com foco em indicadores de eficiência de equipes, status das ordens de serviço e relatórios de desempenho.
- Técnico: Visão estritamente operacional contendo a listagem de ordens de serviço atribuídas, checklists pendentes e cronograma de preventivas do dia.

10. CONCLUSÃO

O MaintSys consolida-se como uma solução robusta para a gestão de manutenção industrial no contexto da Indústria 4.0. A plataforma integra controle operacional, gestão preventiva, fluxo de manutenção corretiva, checklists técnicos e indicadores de desempenho (KPIs), mantendo um histórico completo de todas as atividades. O desenvolvimento deste projeto viabilizou a transição crítica e necessária dos registros físicos para um ecossistema digital moderno, centralizado e seguro.

O alicerce dessa jornada foi a criação pioneira da versão Web. Essa etapa inicial estabeleceu a infraestrutura e a lógica de negócios indispensáveis para viabilizar, posteriormente, a expansão e a implementação da versão mobile.

O sucesso do projeto sustentou-se em três pilares fundamentais:

Evolução Estratégica: A partir da base Web estável, o sistema expandiu-se para o ambiente mobile utilizando a tecnologia FlutterFlow. A aplicação rigorosa da metodologia MoSCoW foi decisiva para priorizar os requisitos de maior valor estratégico, garantindo entregas ágeis e focadas.

Gestão e Arquitetura de Dados: Implementou-se um controle estrito sobre as ordens de serviço, alocação de técnicos e históricos de intervenção. A modelagem de dados foi concebida por meio de diagramas verticais que se converteram diretamente na estrutura do banco de dados. Essa definição precisa e a tipagem correta dos dados logo na fase inicial eliminaram retrabalhos na criação dos schemas do backend.

Usabilidade e Agilidade: A interface, projetada com foco na experiência do usuário, foi inicialmente prototipada no Figma e transposta com agilidade para o ambiente mobile, otimizando o tempo de desenvolvimento. O fluxo de trabalho foi gerenciado via metodologia Kanban, com o quadro organizado por camadas funcionais — uma estratégia que impediu que pendências visuais ou de design bloqueassem o avanço da lógica do sistema.

Diante dessa trajetória de desenvolvimento, o MaintSys cumpre com excelência o desafio proposto. Ao elevar o padrão dos processos de manutenção do Senai, o projeto entrega uma plataforma escalável, madura, de alta performance e estritamente orientada a dados.